



Jormungandr — Canon Governance and Enforcement Layer

Mimir Labs Technical Publication

Document:	ML-WP-011	Version:	1.1
Author:	Christopher Gaither	Date:	June 2026
Classification:	Public		

0.1 Overview

Jormungandr is the governance enforcement layer of the Mimir Labs data platform and the standalone expression of ROPE (Runtime Operational Policy Enforcement) for systems that are not Yggdrasil ERP. Named for the World Serpent that encircles Midgard in Norse mythology, it wraps around an organization's data landscape and detects when structures, integrations, or schema changes diverge from the approved canonical model.

Inside Yggdrasil ERP, ROPE refuses illegal state transitions at the database gate through the State Constraint Engine. Jormungandr is that same authority turned outward: it owns the canonical definitions and the legality of state transitions for external systems, and it emits governance contracts those systems validate against before committing a change. Versioned canon registries, drift detection, and compliance reporting make that enforcement durable over time. Jormungandr is currently in pilot.

Where Ratatosk discovers and proposes canonical meaning, Jormungandr owns and enforces it. Where Ragnarok executes a one-time migration and Bifrost maintains ongoing synchronization, Jormungandr ensures that the semantic foundations underlying those processes remain intact over time.

Jormungandr converts one-time governance extraction engagements into long-term enforcement subscriptions. Without it, the canonical model discovered by Ratatosk decays as systems evolve independently. With it, organizations maintain the semantic discipline that makes migrations, integrations, and operational coherence possible.



0.2 Architectural Role

Jormungandr occupies the final position in the Mimir Labs platform lifecycle.

Ratatosk discovers schema structures, classifies them into business domains, and produces a semantic manifest. Ragnarok consumes that manifest to execute governed data migrations. Bifrost uses the manifest to maintain live synchronization between systems. Jormungandr imports the manifest as the baseline canonical model and continuously validates that enterprise data structures remain compliant.

The relationship between Ratatosk and Jormungandr is particularly important. Ratatosk is an engagement-based discovery tool that produces a point-in-time semantic snapshot. Jormungandr transforms that snapshot into a living governance artifact that evolves under controlled versioning while enforcing compliance across the enterprise stack.

Mimisbrunnr serves as the Rosetta Stone for semantic matching within Jormungandr. Canon enforcement applies to any enterprise system, not exclusively to Yggdrasil ERP.

0.3 Design Principles

Jormungandr is governed by constraints that ensure its governance function remains trustworthy and reproducible.

All matching, classification, and validation logic is deterministic and rule-based. No probabilistic or AI-driven behavior is introduced into the governance pipeline. This ensures that validation results are reproducible and auditable.

The system operates purely on schema structures and business labels. It does not require access to operational data to perform governance analysis. This allows meaningful validation without exposing sensitive production information.

Canon enforcement is system-agnostic. While Mimisbrunnr provides the reference vocabulary, Jormungandr can validate schemas from any enterprise system against the canonical model. The governance function is independent of the operational platform.

0.4 Canon Registry

The canonical schema registry is the foundational data structure within Jormungandr. It stores the authoritative definitions against which all validation is performed.

The registry contains table-level definitions with business labels, domain assignments, and ownership metadata. Column-level definitions capture data types, semantic labels, classifications, and



constraints. Taxonomy groups preserve the domain hierarchy established during Ratatosk discovery. Version history records every change to the canonical model with computed diffs between versions.

All registry data is scoped by tenant with row-level security, ensuring complete isolation between organizations.

Canon versioning follows an immutable model. Previous versions are never modified. Changes create new versions with explicit diffs. Any version can be designated as the active validation target, and rollback to previous versions is supported.

0.5 Manifest Import

The import pipeline accepts Ratatosk manifests as the primary mechanism for establishing and updating canonical definitions.

On first import, the manifest establishes version one of the organizational canon. Subsequent imports from additional Ratatosk engagements produce a diff against the existing canon before merging. Each import records the source engagement identifier, date, and facilitator for traceability. Re-importing an identical manifest is a no-op, ensuring idempotency.

This pipeline connects the discovery phase of Ratatosk directly to the enforcement phase of Jormungandr, creating a continuous governance lifecycle from initial analysis through ongoing compliance.

0.6 Controlled Vocabulary

Jormungandr maintains a synonym dictionary that enables deterministic semantic matching between external schemas and canonical definitions.

The dictionary maps alternative terms to their canonical equivalents. Common enterprise variations such as customer identifier, client identifier, and account number all resolve to a single canonical term. Purchase order, procurement order, and buy order all resolve to the canonical purchase order definition.

Synonyms are scoped per taxonomy domain, acknowledging that a term may carry different meaning in different business contexts. The dictionary ships with a baseline set derived from Ratatosk's synonym groups and can be extended through bulk import or manual editing. Changes to the dictionary are versioned alongside canon changes.



0.7 Schema Validation

The validation engine is the core value proposition of Jormungandr. It detects when data structures diverge from the canonical model.

Incoming schemas can be submitted in multiple formats including DDL statements, JSON schema definitions, CSV headers, and API schema documents. Each submission is parsed into a normalized table and column structure, then compared against the active canonical schema.

Each element in the incoming schema is classified as one of several match types. Exact matches occur when both name and type align with canon. Semantic matches occur when the name resolves through the synonym dictionary. Type mismatches indicate name alignment with incompatible data types. Label mismatches indicate structural alignment with differing business semantics. Elements present in the input but absent from canon represent potential drift. Canonical elements missing from the input represent coverage gaps.

Strictness is configurable per tenant. Policies can auto-accept semantic matches, reject all unknowns, operate in warn-only mode, or apply custom thresholds per severity level.

0.8 Semantic Matching

The matching engine provides deterministic semantic equivalence detection without AI dependency.

Matching employs several complementary techniques. Synonym lookup resolves terms against the controlled vocabulary. Structural similarity analysis uses string distance algorithms and common prefix and suffix detection. Abbreviation expansion converts common shorthand to full canonical terms. Type-aware weighting increases confidence when data types align alongside name matches. Positional heuristics consider column position and surrounding context for additional signal.

Each match receives a confidence classification. High confidence indicates synonym match combined with type match. Medium confidence indicates structural similarity with type alignment. Low confidence indicates structural similarity alone. Low-confidence matches are queued for human review rather than auto-accepted, preventing governance automation from propagating incorrect assumptions.

0.9 Governance Reporting

Jormungandr produces structured reports that make governance posture visible across the organization.

Compliance summaries report the percentage of canonical coverage achieved by validated schemas,



broken down by table and domain. Violation reports provide detailed listings of missing tables, unknown columns, type mismatches, and label conflicts. Drift trend reports track validation results over time, revealing whether compliance is improving or degrading. Per-system breakdowns show compliance per source system when multiple schemas are validated.

Violations are classified by severity. Critical violations indicate structural failures such as missing required tables or incompatible types on key fields. Warnings indicate semantic mismatches where a field exists but its label or type differs from canon. Informational findings flag new fields outside canon that may represent legitimate extensions.

Reports are available in JSON for machine consumption and CI/CD integration, PDF for executive governance committees, and CSV for analyst review. Scheduled generation supports weekly governance digests, monthly compliance trends, and on-demand reports per validation run.

0.10 Multi-Tenant Architecture

Jormungandr is designed as a multi-tenant SaaS platform with full organizational isolation.

All data is scoped by tenant identifier with PostgreSQL row-level security. Configuration, canonical definitions, synonym dictionaries, and validation history are fully independent per tenant.

Authentication uses JWT-based sessions with API key support for programmatic access. Role-based access control defines three operational roles. Governance administrators have full editing authority over canon, synonyms, policies, and users. Analysts can validate schemas, view reports, and review matches. Viewers have read-only access to reports and canonical definitions.

Subscription management tracks tenant status and supports feature gating per plan tier. All tenant actions are audit-logged.

0.11 API Surface

Jormungandr exposes a REST API covering canon management, schema validation, synonym administration, governance reporting, and tenant operations.

All endpoints require authentication via JWT or API key. Role-based access control is enforced per endpoint. The API follows REST conventions with consistent error responses and auto-generated OpenAPI documentation. Rate limiting protects the service on a per-tenant basis.

The API is designed to support both interactive governance workflows and automated CI/CD integration, enabling organizations to validate schema changes as part of their deployment pipeline.



0.12 Business Model

Jormungandr represents the recurring revenue component of the Mimir Labs platform.

Ratatosk engagements produce a one-time fee and a canonical manifest. That manifest is imported into Jormungandr, establishing an ongoing governance subscription. The value proposition is straightforward: without continuous enforcement, the canonical model decays as systems evolve independently. Jormungandr ensures that the investment in governance extraction pays ongoing dividends through continuous drift detection and compliance reporting.

0.13 Platform Significance

Jormungandr transforms governance from a point-in-time activity into a persistent organizational capability.

Most enterprise governance efforts produce documents that decay as soon as systems change. Jormungandr converts those documents into enforceable policy by maintaining a living canonical model, continuously validating incoming structures against it, and reporting compliance status in actionable terms.

Within the broader Mimir Labs architecture, Jormungandr closes the governance loop. Ratatosk discovers meaning. Ragnarok migrates data into aligned structures. Bifrost maintains synchronization. Jormungandr ensures that the semantic foundations underlying all of these processes remain intact.

It does not discover meaning. It does not move data. It enforces the canonical truth that makes discovery, migration, and synchronization trustworthy.

Copyright 2026 Mimir Labs. All rights reserved.

Mimir Labs LLC | Harrisburg, PA

© 2024–2026 Mimir Labs LLC. All rights reserved.

This document contains proprietary information belonging to Mimir Labs LLC. No part of this publication may be reproduced, distributed, or transmitted in any form without the prior written permission of Mimir Labs LLC. The information herein is provided “as is” without warranty of any kind. Product names and trademarks referenced in this document are the property of their respective owners.